Спецификация для гибких запросов

Содержание

1. Решение проблемы отсутствия гибких фильтров 2
   1. 1. Пример спецификации 2
2. Обработка спецификаций в адаптере 2
   2. 1. Создание списков фильтров и параметров 3
   2. 2. Добавление фильтров и параметров 4
   2. 3. Защита от пустой спецификации 4
   2. 4. Формирование SQL-запроса 4
   2. 5. Выполнение запроса 5
3. Итоговая функция 6
4. Функции построения спецификации 8

Задачи 9

1. Решение проблемы отсутствия гибких фильтров

Здесь есть одна проблема. **Отсутствие гибких фильтров** для получения каких-либо данных из БД. Что имеется ввиду? Дело в том, что текущий `get()` от `ItemsSqliteStore` работает с фильтром только по **индентификатору**, а что если нам нужны разнообразные фильтры? Не только по идентификатору, но и по какому-либо другому полю?

Что можно сделать в таком случае? Два варианта:

- Объявлять отдельные функции, например: `get_by_name()`, `get_by_price()` и так далее.
- Создать `Контекст фильтров`, ровно так же, как и мы сделали `Контекст запроса` для HTTP обработчика. На основе этого `Контекста фильтров`, адаптер работы с БД должен построить нужный запрос с нужными фильтрами и выполнить запрос.

Что имеется ввиду? Мы сделаем «коробку» со значениями для фильтрации, которые как могут быть переданы, так и не могут. Если они переданы — нам нужно добавить их для фильтра, если не переданы — значит их добавлять не нужно (например, потому что не требуется фильтрация по какому-то неуказанному полю).

1. 1. Пример спецификации

Создадим в проекте структуру:

```
#[doc = "Спецификация товара, которая определяет
         Какой товар нам нужно достать из базы данных."]
pub struct ItemSpecification {
    pub id: Option<Uuid>,
//  ^^^^^^^^^^^^^^^^^^^^^^^^^^
//  ВОЗМОЖНО ХОТИМ ОТБИРАТЬ ПО ИДЕНТИФИКАТОРУ.
//  Здесь значение может быть а может нет.

    pub name: Option<String>,
//  ^^^^^^^^^^^^^^^^^^^^^^^^^^^
//  ВОЗМОЖНО ХОТИМ ОТБИРАТЬ ПО НАЗВАНИЮ ТОВАРА.
//  Здесь значение может быть а может нет.

    pub price: Option<Price>,
//  ^^^^^^^^^^^^^^^^^^^^^^^^^^^
//  ВОЗМОЖНО ХОТИМ ОТБИРАТЬ ПО ЦЕНЕ ТОВАРА.
//  Здесь значение может быть а может нет.
}
```

Информация:

Это **спецификация** какой-то сущности. ***Она описывает «по каким параметрам» мы хотим найти нужную сущность из нашей базы данных***

2. Обработка спецификаций в адаптере

Теперь, исходя из этого, нам нужно **научить наш адаптер работы с базой данных** обрабатывать спецификации.

Обновим код нашего адаптера работы с базой данных, чтобы он мог принимать `ItemSpecification` и строить запросы с нужными фильтрами.

Было:

```
pub trait ItemsStorage {
//  ДРУГОЙ КОД
//  ..........................

    fn get(
        &self,
        uuid: Uuid,
    ) -> impl Future<Output = Result<(), Box<dyn std::error::Error>>>;
//  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//  РАНЬШЕ ТОЛЬКО ИДЕНТИФИКАТОР

//  ..........................
//  ДРУГОЙ КОД
}
```

Стало:

```
pub trait ItemsStorage {
//  ДРУГОЙ КОД
//  ..........................

    fn get(
        &self,
        specification: ItemSpecification,
    ) -> impl Future<Output = Result<(), Box<dyn std::error::Error>>>;
//  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
//  ТЕПЕРЬ МЫ МОЖЕМ ФИЛЬТРОВАТЬ ПО ЛЮБОМУ ПОЛЮ,
//  УКАЗАННОМУ В `ItemSpecification`

//  ..........................
//  ДРУГОЙ КОД
}
```

Теперь важно это реализовать внутри реализации адаптера работы с базой данных, чтобы метод `get` корректно обрабатывал `ItemSpecification` и строил соответствующие запросы к базе данных.

2. 1. Создание списков фильтров и параметров

Создадим два списка (вектора), один для хранения вариантов фильтров, другой для хранения параметров для этих фильтров:

```
async fn get(
    &self,
    specification: ItemSpecification,
) -> Result<(), Box<dyn std::error::Error>> {
    let mut filters: Vec<&str> = Vec::new();
//      ^^^^^^^^^^^^^^^^^^^
```

```
//      Здесь мы будем добавлять параметры для каждого поля спецификации.
}
```

2. 2. Добавление фильтров и параметров

Дальше используя обычный `if` проверяем каждое поле спецификации и добавляем соответствующие фильтры и параметры:

```
if let Some(id) = specification.id {
    filters.push("id = ?");
    parameters.push(id.into());
}
// ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
// Если в спецификации указано значение для `id`,
// мы добавляем соответствующий фильтр и параметр.

if let Some(name) = specification.name {
    filters.push("name = ?");
    parameters.push(name.into());
}
// ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
// Если в спецификации указано значение для `name`,
// мы добавляем соответствующий фильтр и параметр.

if let Some(price) = specification.price {
    filters.push("price = ?");
    parameters.push(price.into());
}
// ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
// Если в спецификации указано значение для `price`,
// мы добавляем соответствующий фильтр и параметр.
```

2. 3. Защита от пустой спецификации

Добавим защиту, если ни одно поле спецификации не указано, чтобы не выполнять запрос без фильтров:

```
if filters.len() == 0 {
    return Ok(None);
}
```

2. 4. Формирование SQL-запроса

Сформируем SQL-запрос с учетом всех фильтров:

```
let query: String = format!(
    "SELECT
        id,
        name,
        price,
        created_at,
```

```
        updated_at,
        deleted_at
    FROM
        items WHERE 1=1
    AND {}",
    filters.join(" AND ")
);
```

`filters.join(" AND ")` объединяет все фильтры в одну строку, разделяя их логическим оператором `AND`, чтобы корректно вставить их в SQL-запрос.

Например итоговый запрос будет выглядеть так:

```
SELECT
    id,
    name,
    price,
    created_at,
    updated_at,
    deleted_at
FROM
    items
WHERE 1=1
AND id = ? AND name = ? AND price = ?;
```

2. 5. Выполнение запроса

После формирования запроса и параметров, можно выполнить его с использованием соединения с базой данных, передав `query` и `parameters` в метод выполнения запроса:

```
let result = self
    .connection
    .conn(move |connection| {
        let mut statement = connection.prepare(&query)?;
        statement
            .query_one(params_from_iter(parameters), |row| {
                let id: uuid::Uuid = row.get(0)?;
                let name: String = row.get(1)?;
                let price: f64 = row.get(2)?;

                let item_id: EntityId = EntityId::create(id).map_err(|_| {
                    rusqlite::Error::ToSqlConversionFailure(Box::new(
                        std::io::Error::new(
                            std::io::ErrorKind::Other,
                            "Failed to create EntityId",
                        )
                    ))
                })?;

                let name: Name = Name::new(name).map_err(|_| {
                    rusqlite::Error::ToSqlConversionFailure(Box::new(
                        std::io::Error::new(
                            std::io::ErrorKind::Other,
```

```
                            "Failed to create Name",
                        )
                    ))
                })?;

                let price: Price = Price::new(price).map_err(|_| {
                    rusqlite::Error::ToSqlConversionFailure(Box::new(
                        std::io::Error::new(
                            std::io::ErrorKind::Other,
                            "Failed to create Price",
                        )
                    ))
                })?;

                let lifetime: RecordLifetime =
                    parse_record_lifetime(row, 3, 4, 5)?;
                Ok(Item::restore(ItemId(item_id), name, price, lifetime))
            })
            .optional()
    })
    .await?;

Ok(result)
```

3. Итоговая функция

Итоговая функция выглядит следующим образом:

```
async fn get(
    &self,
    specification: ItemSpecification,
) -> Result<Option<Item>, Box<dyn std::error::Error>> {
    let mut filters: Vec<&str> = Vec::new();
    let mut parameters: Vec<Value> = Vec::new();

    if let Some(id) = specification.id {
        filters.push("id = ?");
        parameters.push(id.into());
    }

    if let Some(name) = specification.name {
        filters.push("name = ?");
        parameters.push(name.into());
    }

    if let Some(price) = specification.price {
        filters.push("price = ?");
        parameters.push(price.into());
    }

    if filters.len() == 0 {
        return Ok(None);
```

```
    }

    let query: String = format!(
        "SELECT
            id,
            name,
            price,
            created_at,
            updated_at,
            deleted_at
        FROM
            items WHERE 1=1
        AND {}",
        filters.join(" AND ")
    );

    let parameters = parameters;

    let result = self
        .connection
        .conn(move |connection| {
            let mut statement = connection.prepare(&query)?;
            statement
                .query_one(params_from_iter(parameters), |row| {
                    let id: uuid::Uuid = row.get(0)?;
                    let name: String = row.get(1)?;
                    let price: f64 = row.get(2)?;

                    let item_id: EntityId = EntityId::create(id).map_err(|_| {
                        rusqlite::Error::ToSqlConversionFailure(Box::new(
                            std::io::Error::new(
                                std::io::ErrorKind::Other,
                                "Failed to create EntityId",
                            )
                        ))
                    })?;

                    let name: Name = Name::new(name).map_err(|_| {
                        rusqlite::Error::ToSqlConversionFailure(Box::new(
                            std::io::Error::new(
                                std::io::ErrorKind::Other,
                                "Failed to create Name",
                            )
                        ))
                    })?;

                    let price: Price = Price::new(price).map_err(|_| {
                        rusqlite::Error::ToSqlConversionFailure(Box::new(
                            std::io::Error::new(
                                std::io::ErrorKind::Other,
                                "Failed to create Price",
                            )
                        ))
```

```
                    })?;

                    let lifetime: RecordLifetime =
                        parse_record_lifetime(row, 3, 4, 5)?;
                    Ok(Item::restore(ItemId(item_id), name, price, lifetime))
                })
                .optional()
        })
        .await?;

    Ok(result)
}
```

4. Функции построения спецификации

Дополнительно, рекомендую добавить для спецификации функции её построения, чтобы можно было создавать спецификации более гибко.

```
impl ItemSpecification {
    pub fn new() -> Self {
        Self {
            name: None,
            price: None,
            lifetime: None,
        }
    }

    pub fn with_name(mut self, name: &str) -> Self {
        self.name = Some(name.to_string());
        self
    }

    pub fn with_price(mut self, price: f64) -> Self {
        self.price = Some(price);
        self
    }

    pub fn with_id(mut self, id: Uuid) -> Self {
        self.id = id;
        self
    }
}
```

Задачи

> Выполняйте задания для сущностей и сценариев своей предметной области. `ItemSpecification` — пример подхода, а не обязательная структура для каждого проекта.

Задача 1. Определить поля поиска

Определите, по каким полям нужно искать сущности в вашем приложении. Опирайтесь на реальные сценарии: поиск записи по идентификатору, проверку существования записи с определёнными данными или отбор по сочетанию нескольких признаков.

Задача 2. Создать спецификации

Создайте спецификации для сущностей, которым требуется гибкий поиск. Объявите необязательные фильтры через `Option` и выберите типы полей в соответствии с вашей моделью и SQL-конвертерами. Добавьте конструктор пустой спецификации и методы построения `with_...` для поддерживаемых фильтров.

Задача 3. Обновить контракты и адаптеры

Обновите контракты и реализации адаптеров: методы поиска должны принимать спецификацию вместо одного фиксированного фильтра.

Задача 4. Реализовать построение SQL-запроса

Реализуйте построение SQL-запроса по заполненным полям спецификации. Объединяйте указанные условия через `AND`, а значения передавайте отдельно через параметры запроса. Не вставляйте пользовательские значения в текст SQL с помощью `format!` или конкатенации строк.

Задача 5. Определить поведение пустой спецификации

Явно определите поведение пустой спецификации. Для поиска одной записи используйте защиту из примера: возвращайте `Ok(None)` без выполнения запроса. Получение всех записей, если оно требуется приложению, оформите как отдельный предусмотренный сценарий.

Задача 6. Применить и проверить спецификации

Примените спецификации в бизнес-логике или HTTP-обработчиках своего проекта. Проверьте поиск по одному фильтру, сочетанию фильтров, отсутствие совпадений и пустую спецификацию. Убедитесь, что при сочетании фильтров результат удовлетворяет всем условиям, а ошибки БД не подменяются отсутствием записи. Исправьте ошибки компиляции, возникшие после изменения контрактов.

Рекомендуется выполнить следующий этап: Конвертация типов вашего приложения в SQL-типы и наоборот